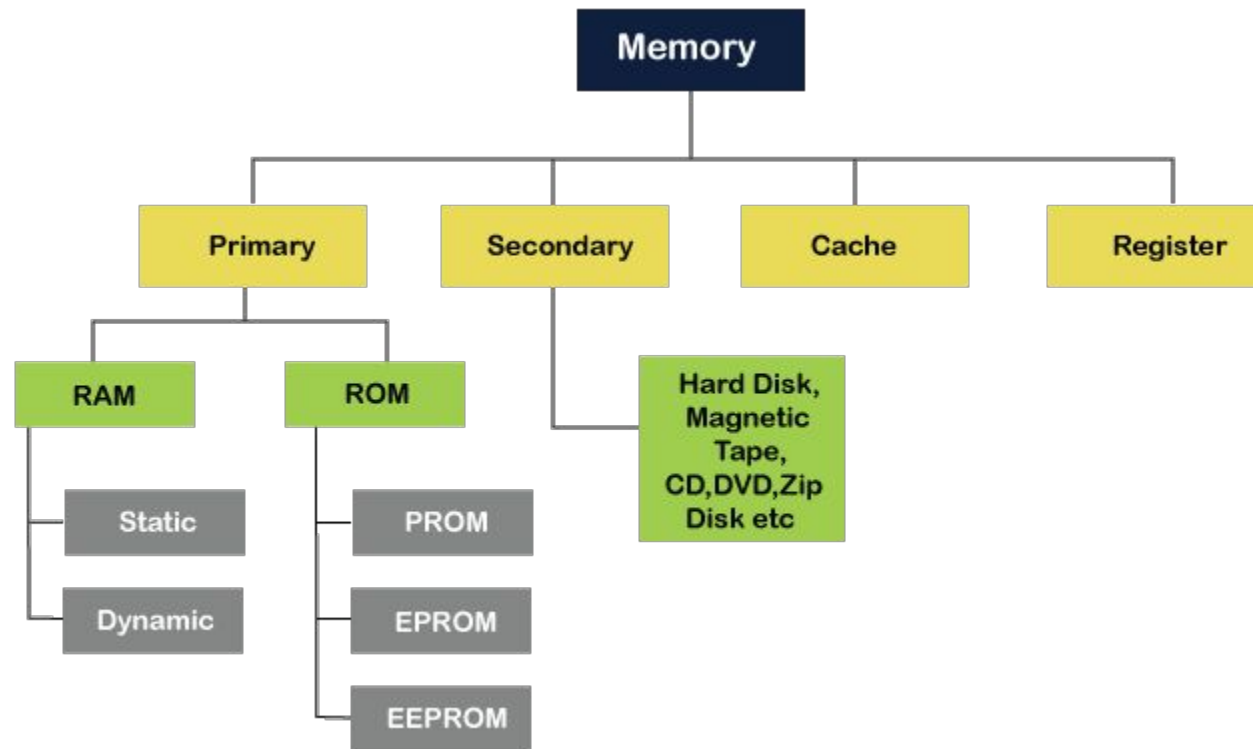


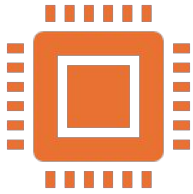
Memory Organization

By : Bilal Ahmed

Classification of Memory



Characteristics of Computer Memory



Capacity

Refers to the amount of data that a memory unit can store.

Bit: The basic unit of memory, representing either 0 or 1.

Byte: The fundamental storage unit, typically consisting of 8 bits.



Internal Memory Capacity

Measured in bytes (8 bits) or in **words**.

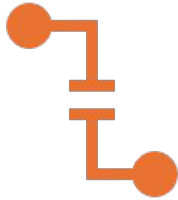
Word size commonly includes 8, 16, or 32 bits depending on the architecture.



External Memory Capacity

Typically expressed in bytes.

Unit of transfer



Defined as the number of electrical lines connected to and from the memory module.

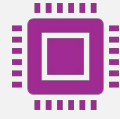


Often equal to the word length, but may be larger (e.g., 64, 128, or 256 bits).

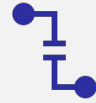


Data transfers can occur in blocks consisting of multiple words.

Unit of transfer



For **main memory (RAM)** → usually a **word** (or multiple words in a block).



For **cache** → typically a **block/line**.



For **disk** → a **sector or block** (commonly 512 bytes or 4 KB).



For **tape** → a **block of records** (streamed sequentially).

Memory Access

Sequential Access: Records are accessed in a fixed linear order; access time varies (e.g., tape).

Direct Access: Blocks have unique addresses; access uses positioning plus sequential steps; variable time (e.g., disk).

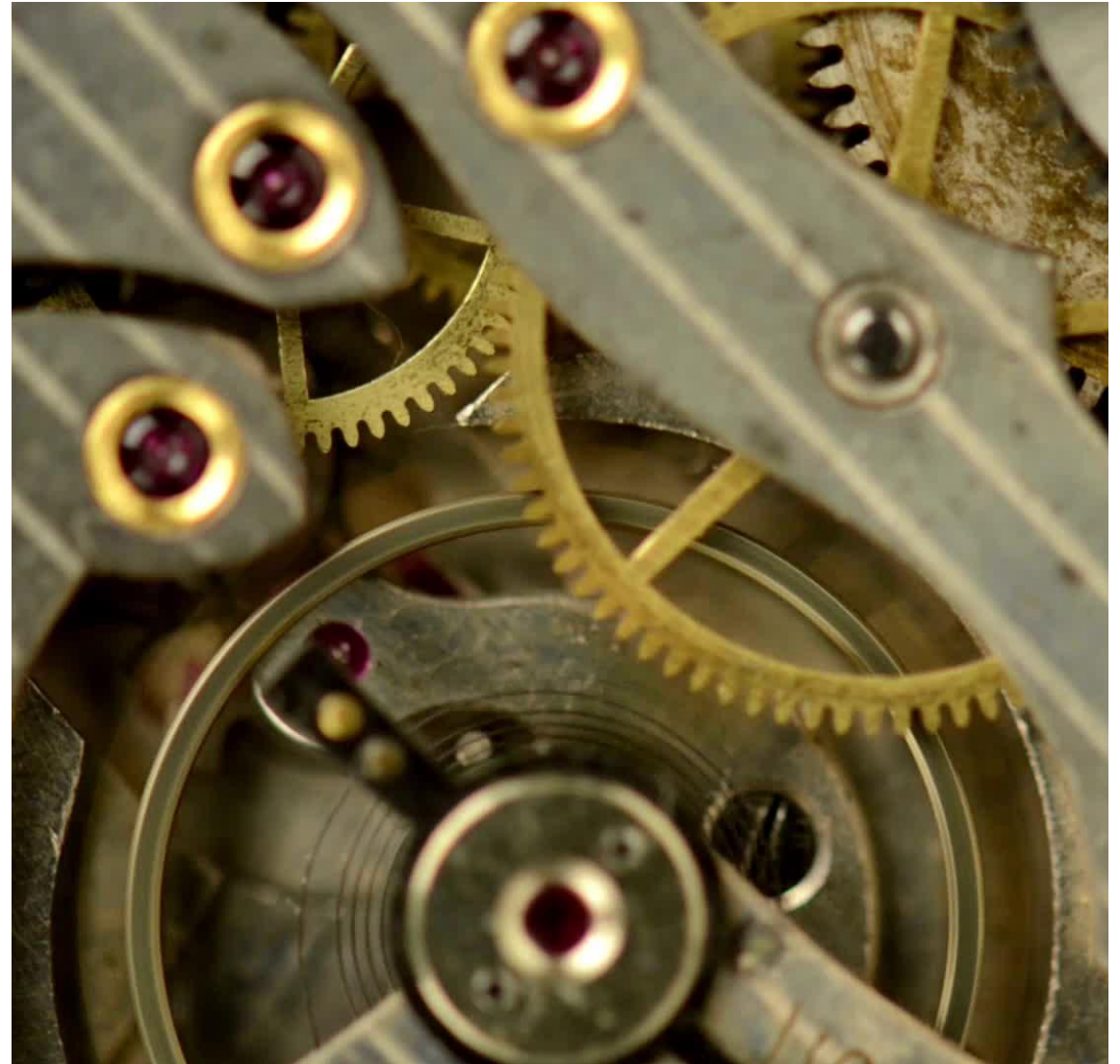
Random Access: Each location directly addressable; constant access time, independent of prior accesses (e.g., main memory, cache).

Summary

Feature	Tape	Disk (HDD/SSD/Optical)	RAM (Main Memory)
Access Method	Sequential Access – records read in linear order, time depends on position.	Direct Access – system seeks to block (random jump), then may read sequentially.	Random Access – any location directly accessed with constant time.
Speed	Slowest	Faster than tape (HDD slower, SSD faster).	Fastest (nanoseconds).
Cost per Bit	Lowest	Moderate	Highest
Capacity	Very high	High	Limited
Volatility	Non-volatile	Non-volatile	Volatile
Use	Backup, archival storage	OS, applications, general file storage	Temporary active data/programs

Memory Performance Parameters: Access Time

- **For Random Access Memory:**
 - Time taken for a read or write operation
 - Time from the instant that an address is presented to the time instant the data is stored or made available for use
- **For Non-Random Access Memory:**
 - Time to position read-write mechanism at desired location



Memory Performance Parameters

- **Memory Cycle Time:**
 - Access Time+ Wait time (regenerate signals)
- **Transfer Rate:**
 - The rate at which data is transferred in/out of the memory
 - For Random-access memory, it is: $1/(\text{cycle time})$



Characteristics of Computer Memory

- **Transfer Rate for Non-Random Access Memory:**

$$T_n = T_A + \frac{n}{R}$$

where

T_n = Average time to read or write n bits

T_A = Average access time

n = Number of bits

R = Transfer rate, in bits per second (bps)



Problem (RAM)

- Find out the time taken for write operation to take place in RAM, when memory cycle time is 5 seconds
-

Problem (Not Random)

- Find out the Transfer Rate for a Solid State Drive
- When unit to read is 16 bits,
- Average access time is 0.001 seconds
- and average time to read n bits is 5 seconds



Analyzing data

We are given:

- Average access time:

$$T_A = 0.001 \text{ seconds}$$

- Average time to read n bits:

$$T_n = 5 \text{ seconds}$$

- Number of bits:

$$n = 16 \text{ bits}$$

The general formula for the average time to read or write n bits is:

$$T_n = T_A + \frac{n}{R}$$

where R is the transfer rate in bits per second.

Solution

Step 1: Substitute the given values

$$5 = 0.001 + \frac{16}{R}$$

Step 2: Rearrange the equation

$$5 - 0.001 = \frac{16}{R}$$

$$4.999 = \frac{16}{R}$$

Step 3: Solve for R

$$R = \frac{16}{4.999}$$

$$R \approx 3.2 \text{ bits per second}$$

Summary

Table 4.1 Key Characteristics of Computer Memory Systems

Location	Performance
Internal (e.g., processor registers, cache, main memory)	Access time
External (e.g., optical disks, magnetic disks, tapes)	Cycle time
	Transfer rate
Capacity	Physical Type
Number of words	Semiconductor
Number of bytes	Magnetic
	Optical
Unit of Transfer	Magneto-optical
Word	Physical Characteristics
Block	Volatile/nonvolatile
Access Method	Erasable/nonerasable
Sequential	Organization
Direct	Memory modules
Random	
Associative	

Memory Design Constraints



Capacity (How much?): More memory enables more demanding applications.



Speed (How fast?): Memory must keep pace with the processor to avoid delays.



Cost (How expensive?): Must be balanced and reasonable compared to other system components.

Memory Trade-offs



Trade-off in memory design: Capacity, speed, and cost cannot all be maximized together.



Faster access → Higher cost per bit



Larger capacity → Lower cost per bit



Larger capacity → Slower access time



Design dilemma: Need both large, low-cost memory and small, fast memory for performance.

Cost and Performance in Memory Hierarchy



Cost per bit decreases as we move down (registers → cache → RAM → disk).



Capacity increases with lower levels (disks hold much more than cache).

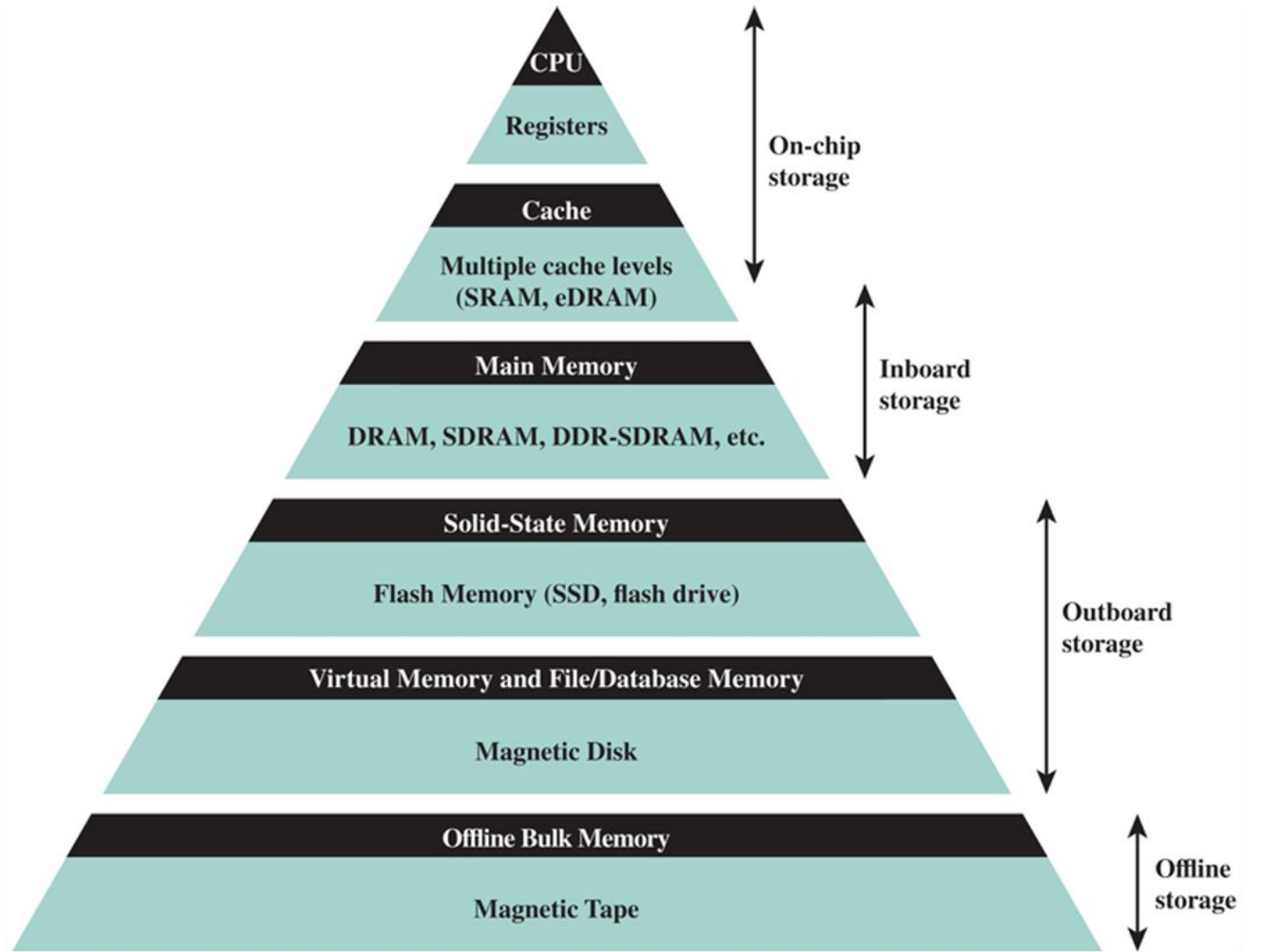


Access time increases (registers are fastest, disks are slowest).

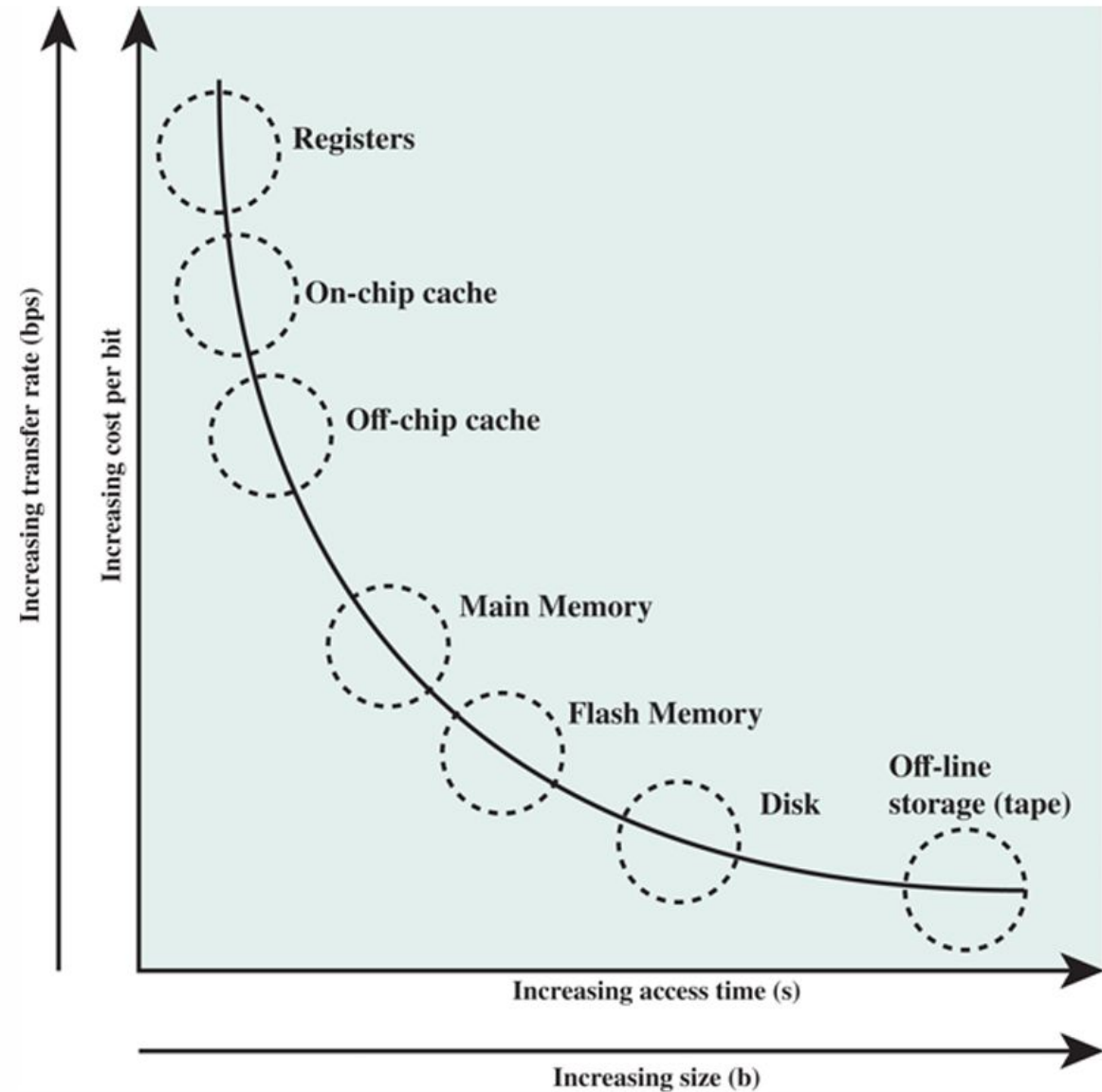


Frequency of access decreases (CPU uses registers and cache most, disk least).

Memory Hierarchy



Cost, Size and Performance in Memory Hierarchy

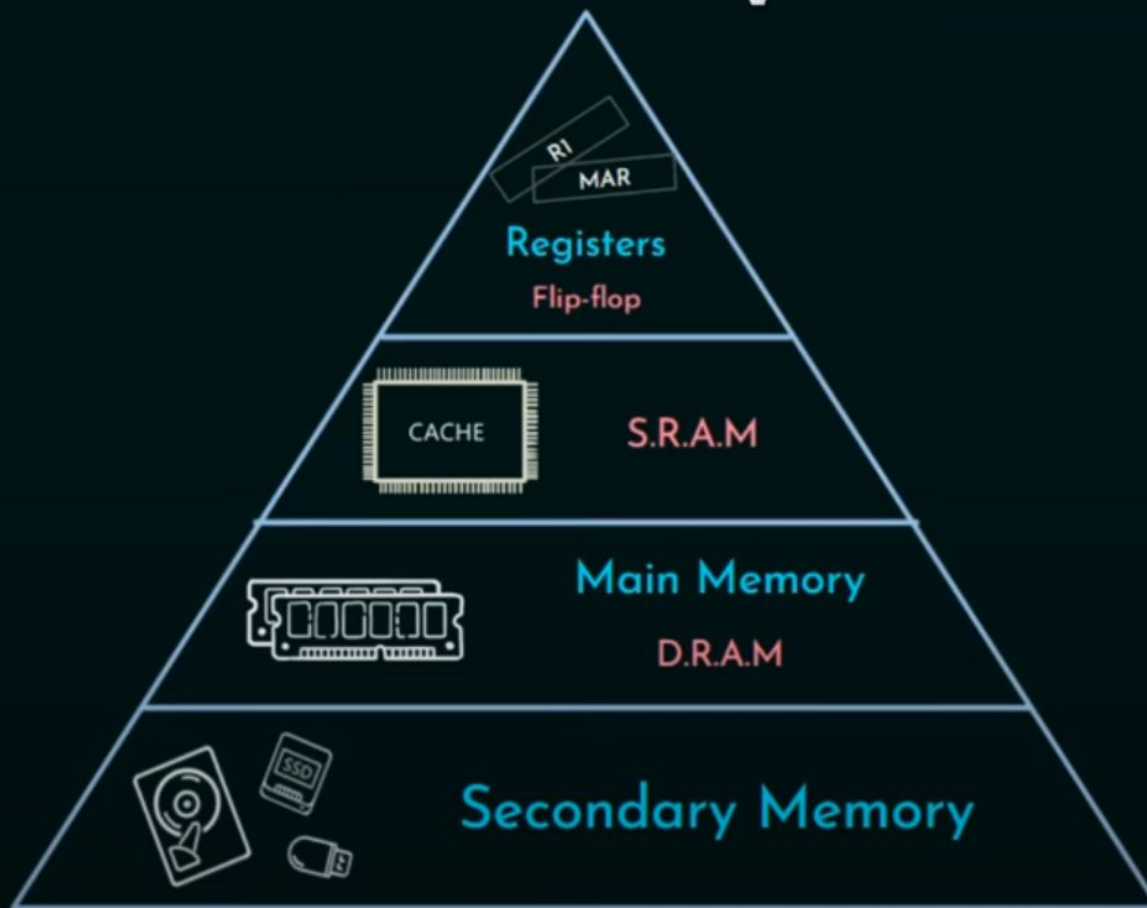


Access
Time

Size

Cost

Usage
Frequency



Cache Memory

Cache Memory: Bridging Speed and Capacity

Cache bridges fast CPU and slower main memory.

Stores small, frequently used data blocks.

Cache hit: quick data access; **Cache miss:** block fetched from main memory.

Locality ensures fetched blocks are likely reused.

Cache Hit/Miss Logic

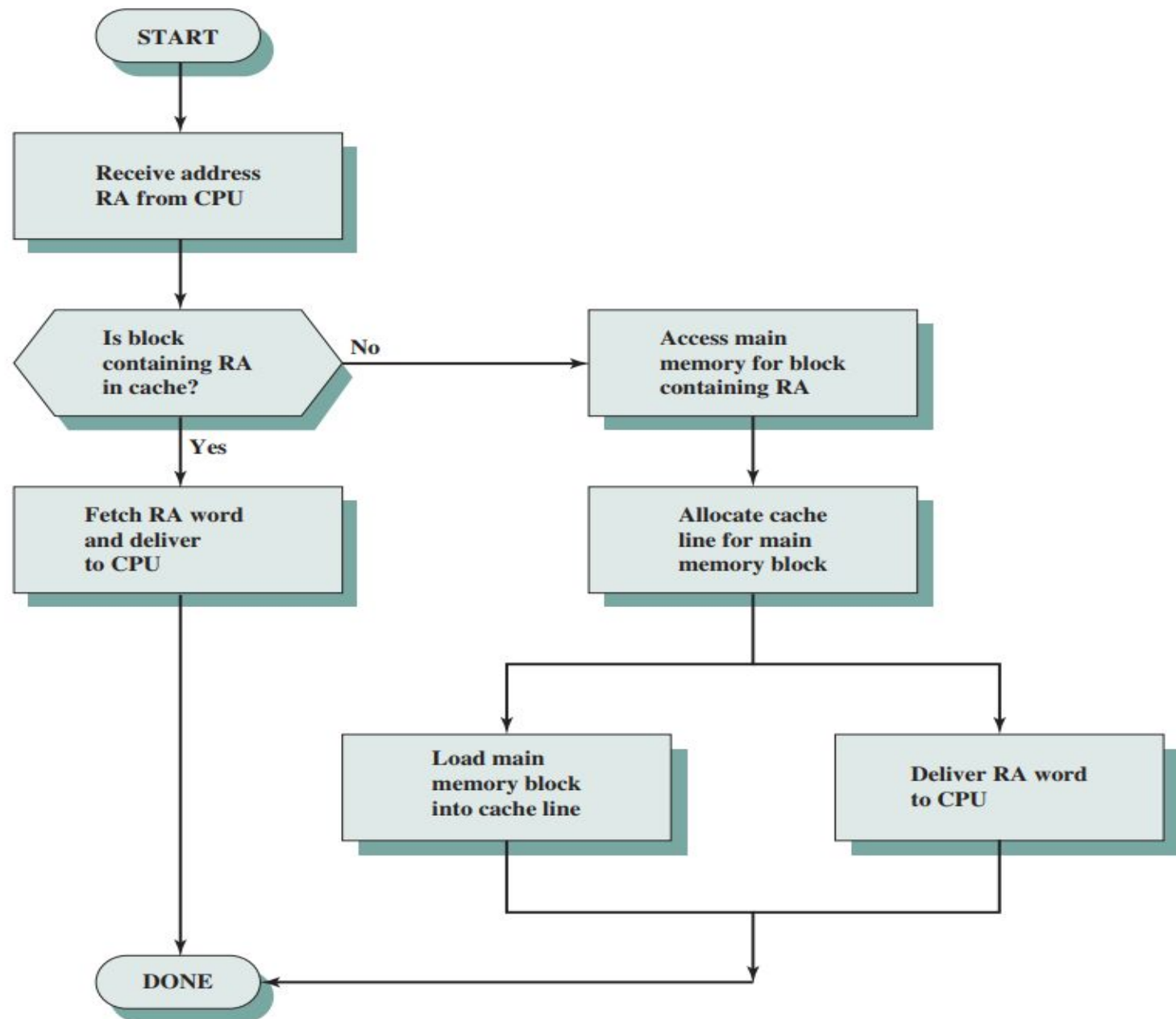
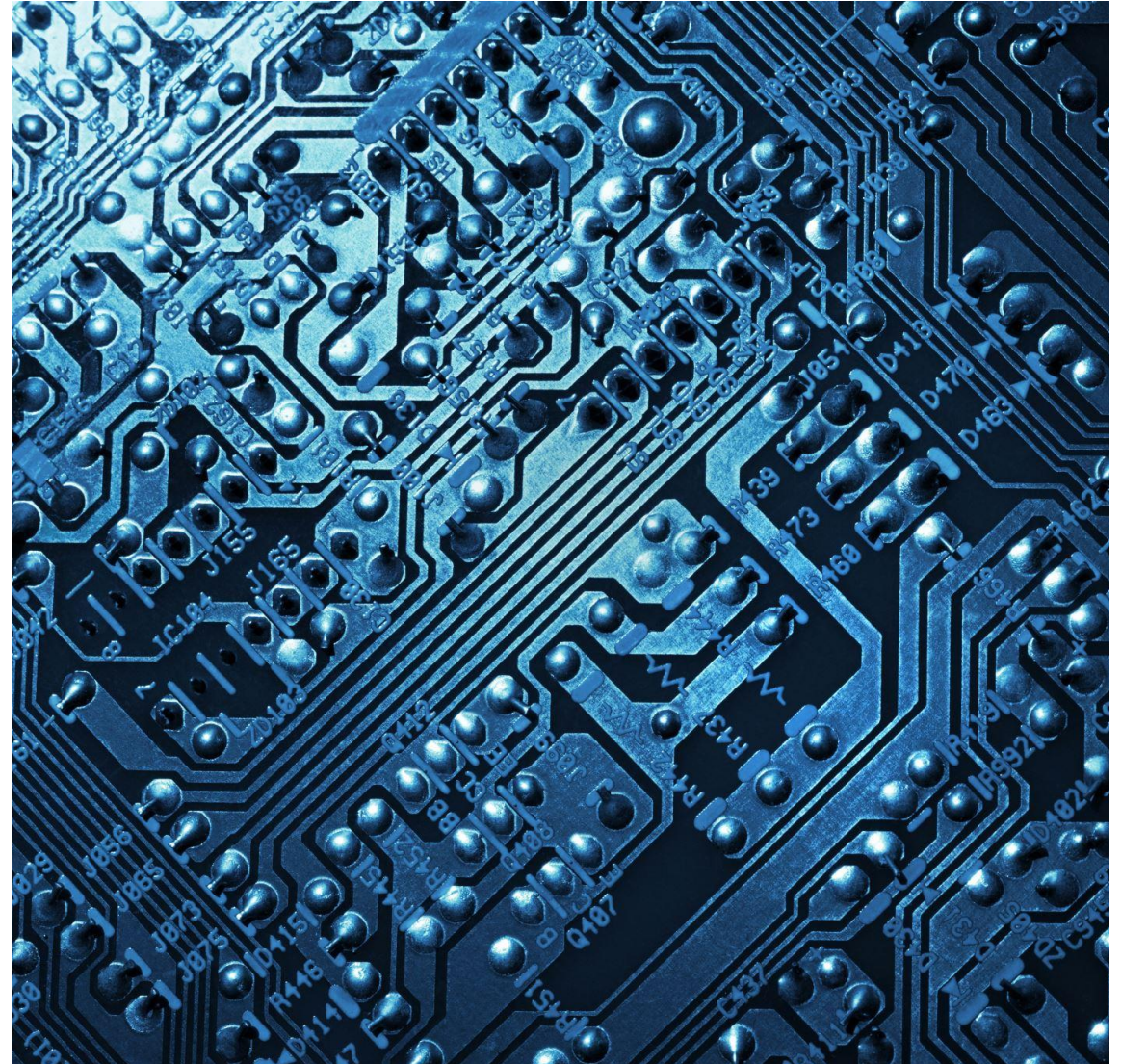


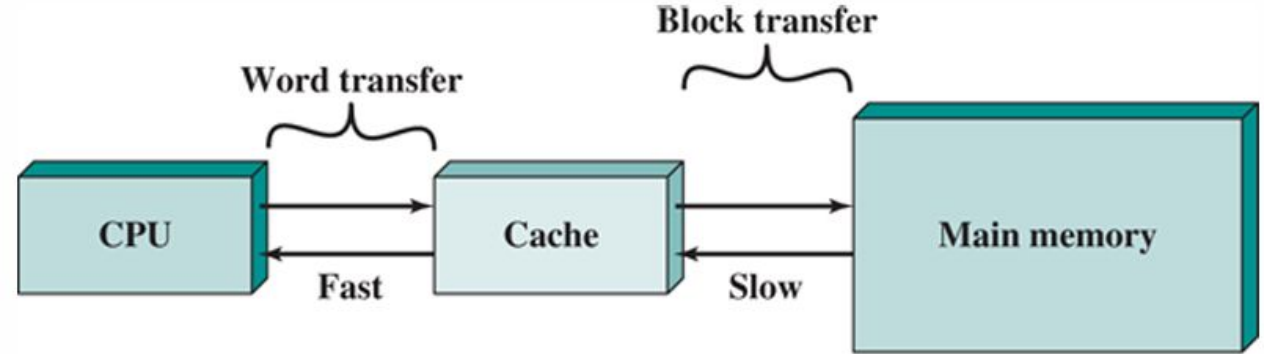
Figure 4.5 Cache Read Operation

Principle of Locality of reference

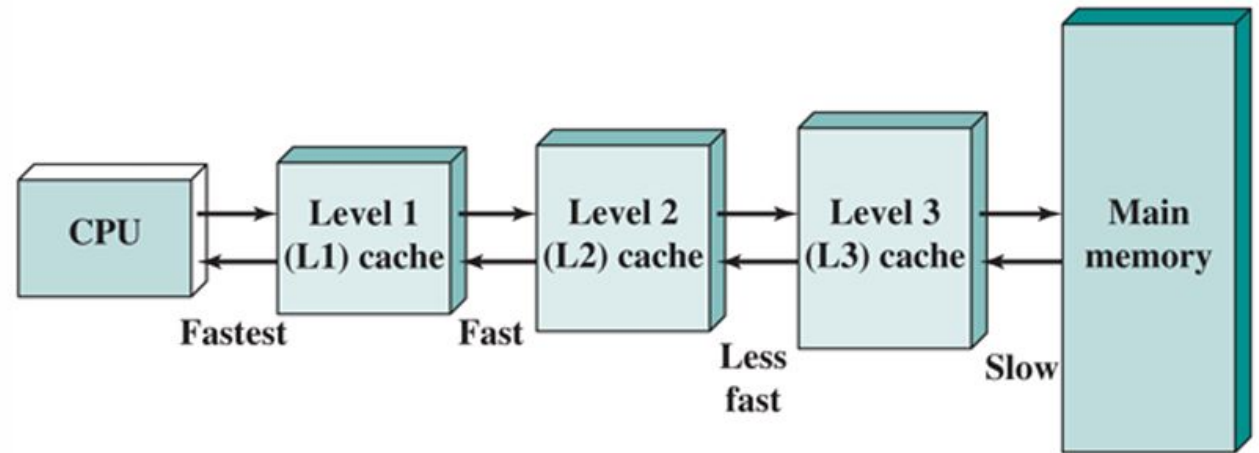
- A processor executes a program:
- A program has arrays, loops etc
- Inside a loop there are instructions which repeat
- Same reference is used for a particular period of time
- It makes sure frequently accessed instructions are in faster memory
- Improves the hit ratio



Cache Levels



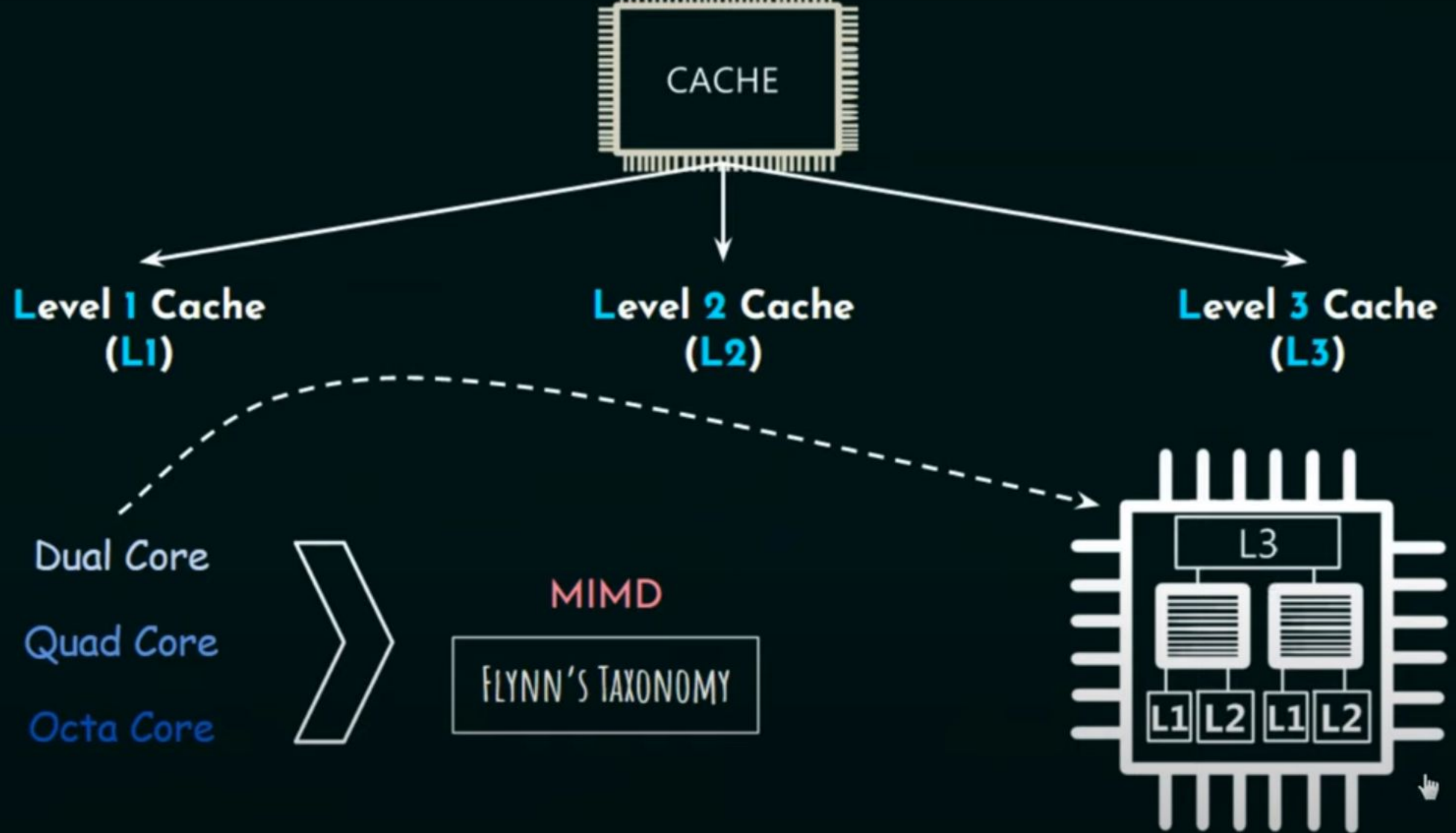
(a) Single cache



(b) Three-level cache organization

Cache Levels

Feature	Single Cache	Three-Level Cache (L1, L2, L3)
Structure	One cache layer between CPU and RAM	Three hierarchical layers (L1, L2, L3)
Speed	Fast (but limited to one layer)	L1 fastest, L3 slowest, but all faster than RAM
Capacity	Limited	Larger total capacity (L1 small, L3 large)
Hit Rate	Lower, since only one cache layer	Higher, as missed data can still be found in other cache levels
Cost	Cheaper, simpler design	More expensive and complex
Use Case	Small/simple CPUs, microcontrollers	Modern CPUs (e.g., Intel i-series, AMD Ryzen)





Flynn's Taxonomy

- It is a classification of **computer architectures** based on the number of **instruction streams** and **data streams** they can handle simultaneously.
-



Flynn's Taxonomy

- SISD**: Single processor executes one instruction on one data stream.
 - SIMD**: Multiple processors execute the same instruction on different data streams.
 - MISD**: Multiple processors execute different instructions on the same data stream.
 - MIMD**: Multiple processors execute different instructions on different data streams.
-



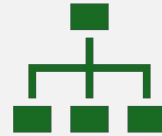
Summary

Category	Instruction Stream	Data Stream	Example
SISD	Single	Single	Old PCs, uniprocessors
SIMD	Single	Multiple	GPUs, vector processors
MISD	Multiple	Single	Rare, fault-tolerant systems
MIMD	Multiple	Multiple	Multicore CPUs, clusters

Structure of Main Memory System



N is the length of each word in bits, so total $2^N - 1$ words can be addressed



Further organization can be converted to blocks:



Where each block may contain K Words



Main Memory Size : 64 words i.e. (0, 1, ..., 63)

Block Size : 4 words

No. of Blocks in MM : $64 / 4 = 16$



64 words

$$\log_2 64 = \log_2 2^6 \\ = 6 \text{ bits}$$

0	0	1	2	3
1	4	5	6	7
2	8	9	10	11
3	12	13	14	15
4	16	17	18	19
5	20	21	22	23
6	24	25	26	27
7	28	29	30	31
8	32	33	34	35
9	36	37	38	39
10	40	41	42	43
11	44	45	46	47
12	48	49	50	51
13	52	53	54	55
14	56	57	58	59
15	60	61	62	63



Cache Terminology



Block → The unit of data transferred between cache and main memory.



Frame → The physical slot in cache or memory that can hold a block (term used for clarity).



Line → A storage location in cache that holds one block plus control info.

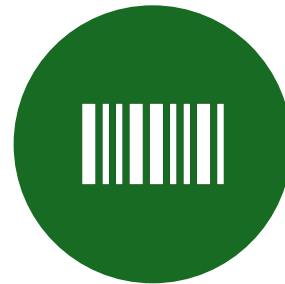


Tag → Address part in a cache line used to identify which block is stored.

Structure of Cache System



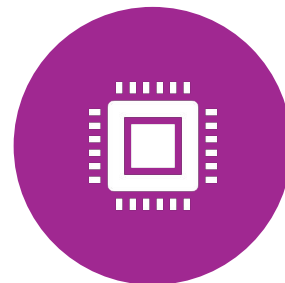
Cache consists of m blocks called LINES, each line may contain K Words, plus a tag.



Control Bits: Indicate if the line has been modified after being loaded.

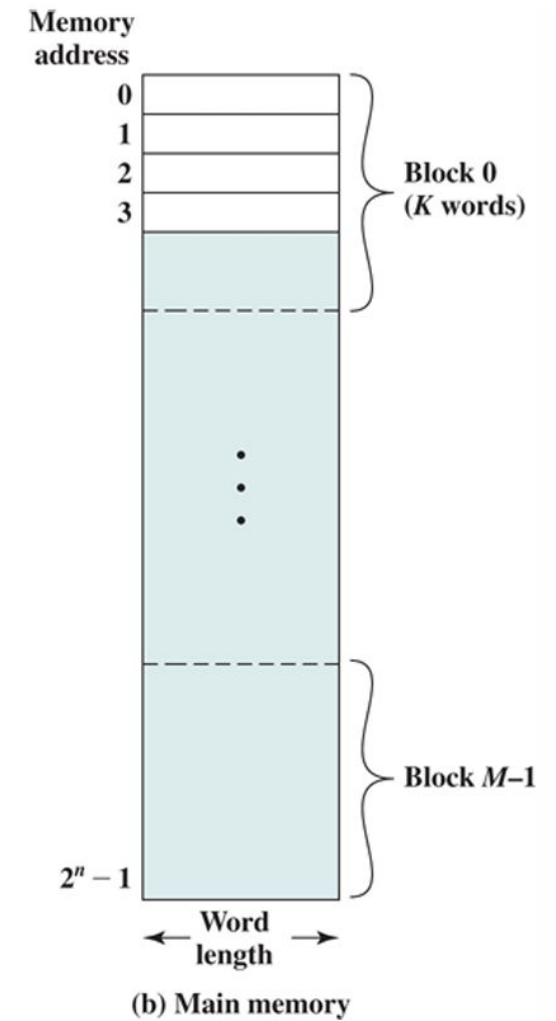
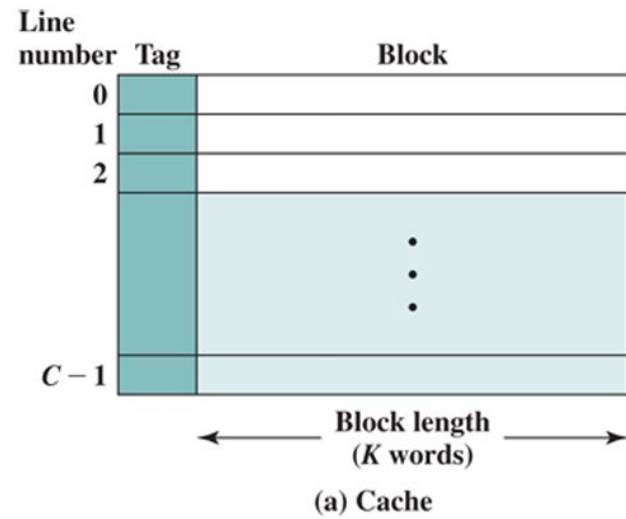


Line Size: The data length of the line (excluding control and tag bits).



Typical Size: Can be as small as 32 bits (e.g., 4 words of 1 byte each).

Structure of a cache/main- memory system



Cache Structure

Cache–Processor Interaction

- **Connections:** Cache connects to the processor through data, address, and control lines.
- **Buffers:** Data and address lines also connect to buffers, which link to the system bus for main memory access.

Cache Hit

- Buffers disabled.
- Direct communication between processor and cache.

Cache Miss

- Address placed on system bus.
- Data fetched from main memory via buffers.
- Word delivered to both cache and processor.

Cache Organization

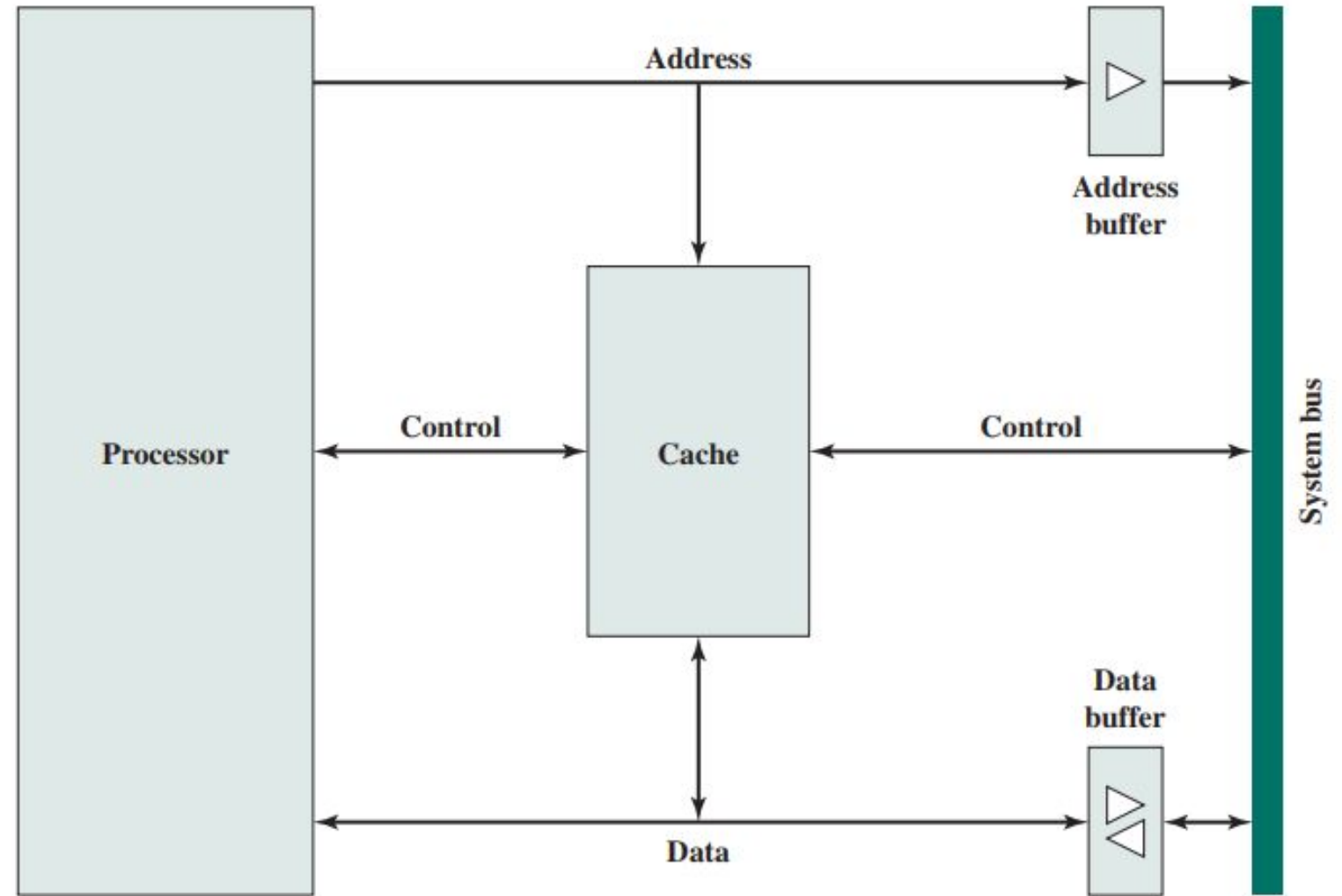


Figure 4.6 Typical Cache Organization

Elements of Cache Design

Cache Addressing & Virtual Memory



Modern Systems: Most computers use virtual memory.



Virtual Address: The address field in instructions refers to a virtual address.



Challenge: CPU must access physical memory for actual read/write operations.

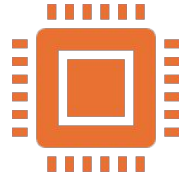


Solution – MMU (Memory Management Unit):



Hardware component that **translates virtual addresses** into physical addresses.

Logical vs Physical Cache



Logical (Virtual) Cache

Uses **virtual** addresses

Accessed **directly** by
processor (no MMU
translation)

Faster access



Physical Cache

Uses **physical** addresses

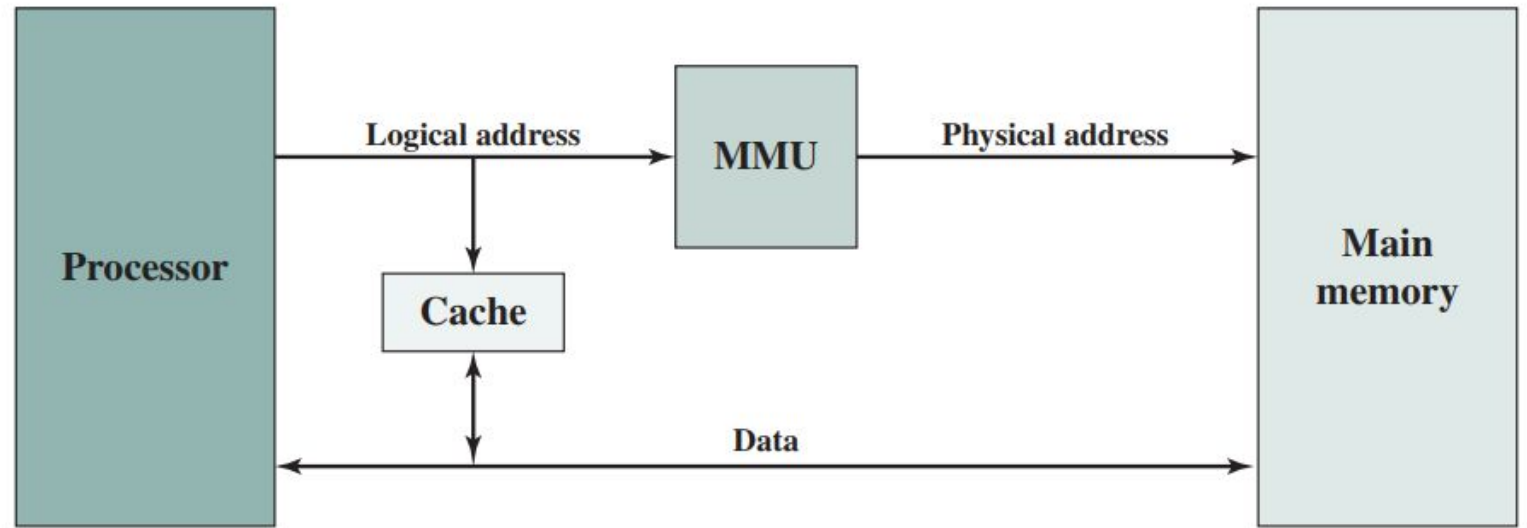
Requires MMU translation
before access

Slower compared to logical
cache

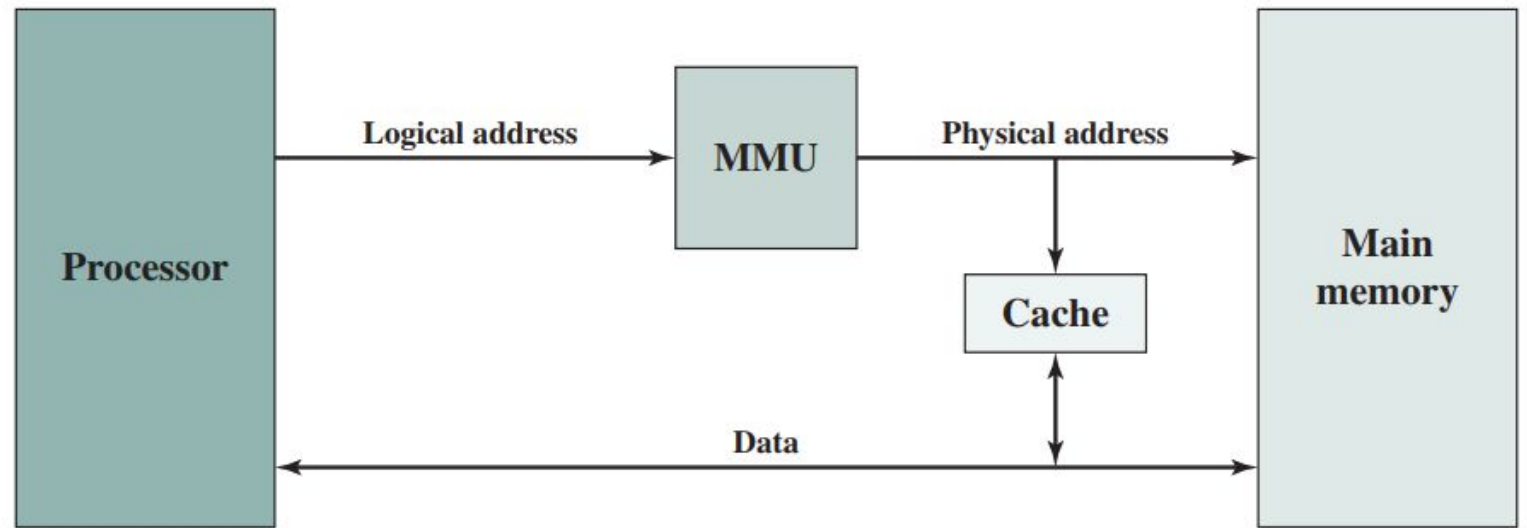
Logical vs Physical Cache

- The **MMU (Memory Management Unit)** translates virtual addresses into physical addresses in RAM.
 - Cache placement choice:**
 - Logical (Virtual) cache** → sits before the MMU, uses virtual addresses.
 - Physical cache** → sits after the MMU, uses physical addresses.
 - Logical cache advantage:** Faster, because it skips waiting for address translation.
 - Logical cache disadvantage:** Since all programs see the same virtual address space (starting at 0), the same VA in two programs may map to different PAs.
-

MMU



(a) Logical cache



(b) Physical cache

Figure 4.7 Logical and Physical Caches

Mapping Function

Since **cache has fewer lines** than memory blocks, a **mapping algorithm** is required.



The mapping function determines:

How **memory blocks** are placed into cache lines.

Which **memory block** currently occupies a given cache line.

Direct Mapping

Each block of memory is mapped to **exactly one specific cache line**.

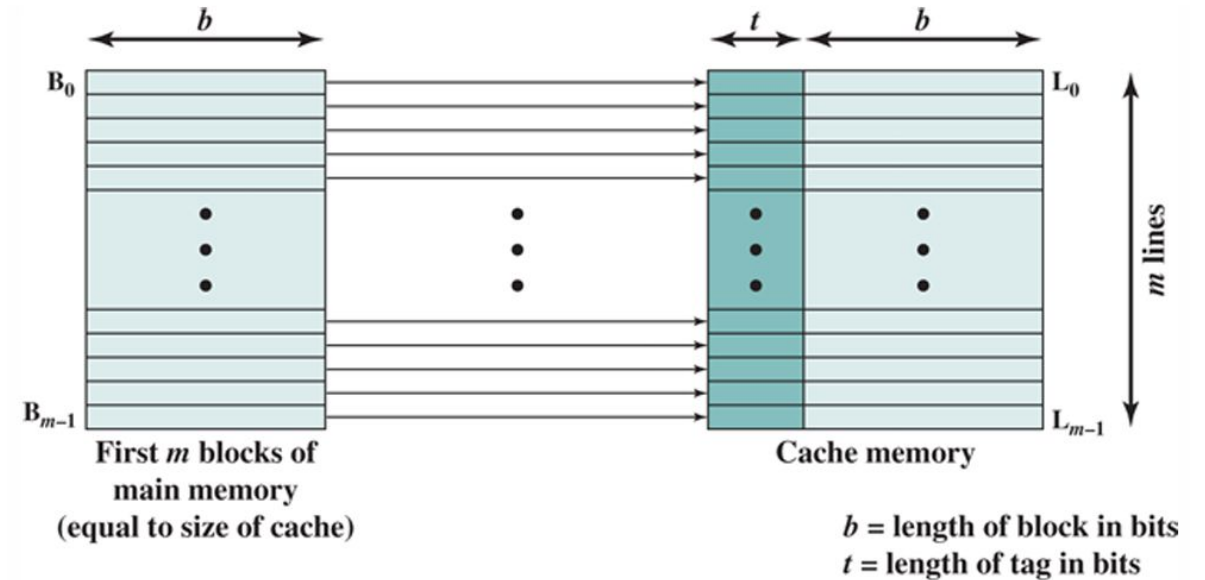
Formula: Cache Line Number = (Block Address) MOD (Number of Cache Lines)

Address is divided into:

- **Tag** – identifies which block of memory is currently in the cache line.
- **Line Index** – selects the cache line.
- **Block Offset** – selects the word within the block.

Example

- Suppose cache has 8 lines.
- Block 12 $\rightarrow 12 \bmod 8 = 4$, so it must go to **line 4**.
- Block 20 $\rightarrow 20 \bmod 8 = 4$, it also goes to **line 4** $\rightarrow$ causes replacement.
- **Key idea:** Very simple, but has lots of conflicts.



(a) Direct mapping

Direct-Mapped Cache

- **Rule:** Each **block of main memory** can go to exactly **one location** in the cache.
 - Example: Suppose cache has 8 lines.
 - Memory block 0, 8, 16 ... → must go to cache line 0.
 - Memory block 1, 9, 17 ... → must go to cache line 1.
 - **Problem:** If two frequently used memory blocks map to the same cache line, they will keep evicting each other → **conflict misses**.
-

Associative Mapping

Any block of memory can be loaded into **any cache line**.

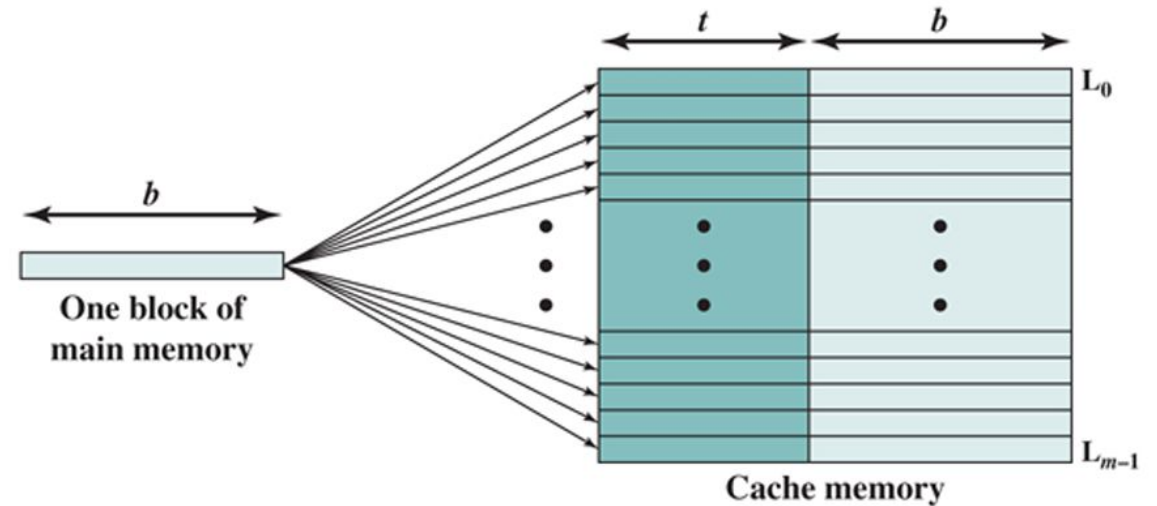
Address is divided into:

- **Tag** – compared against all tags in cache simultaneously.
- **Block Offset** – selects the word within the block.

Cache must search all lines in parallel to check if the block is present.

Example

- Cache has 8 lines.
- Block 12 can be stored in **any of the 8 lines**.
- The cache keeps a **tag** with each line to know which block is there.
- **Key idea:** Flexible, fewer conflicts, but expensive hardware.



(b) Associative mapping



Fully Associative Cache

- **Rule:** Any memory block can be placed **anywhere in the cache**.
 - The cache checks all lines in parallel to see if the block is there.
 - **Benefit:** No fixed mapping → fewer conflicts, because the block can always go into a free or least-recently-used slot.
 - **Problem:** Needs complex hardware (expensive comparators).
-

Set-Associative Mapping

Cache is divided into **sets**, each containing multiple lines (e.g., 2-way, 4-way).

A block maps to a **specific set** (using modulo operation).

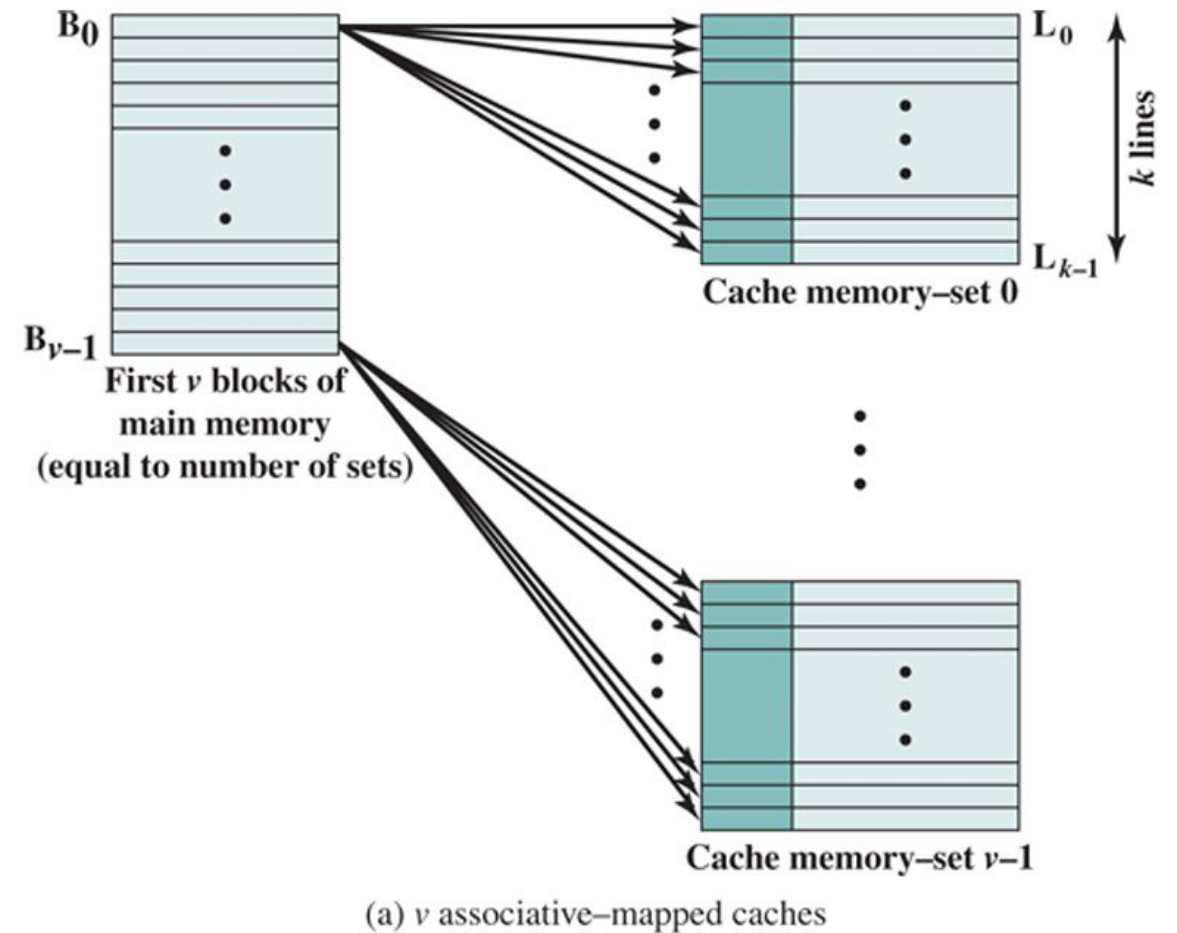
Within that set, the block can occupy **any line**.

Address is divided into:

- **Tag** – identifies the block within the set.
- **Set Index** – selects the set.
- **Block Offset** – selects the word within the block.

Example

- Cache has 8 lines, organized as 4 sets with 2 lines each (2-way).
- Block 12 $\rightarrow (12 \bmod 4) = 0$, so it must go to **Set 0**, but can occupy either of the 2 lines in that set.
- Block 20 $\rightarrow (20 \bmod 4) = 0$, it also goes to **Set 0**, and can share those 2 lines.
- **Key idea:** Balance between simplicity and flexibility.





Set-Associative Cache (middle ground)

- Cache is divided into sets, each with a few lines (2-way, 4-way, etc.).
 - A block maps to a set, but within the set it can go into **any line**.
 - Reduces conflicts compared to direct mapping, but cheaper than full associative.
-

Cache Mapping Comparison

Mapping Type	Placement Rule	Pros	Cons
Direct Mapping	Block $\rightarrow$ One fixed cache line	Simple, low cost	High conflict misses
Associative Mapping	Block $\rightarrow$ Any cache line	Flexible, low conflict	Expensive hardware, slower search
Set-Associative	Block $\rightarrow$ One set, any line within the set	Balanced performance and cost	More complex than direct mapping

Memory

Main Memory Size : 64 words i.e. (0, 1, ..., 63)

Block Size : 4 words

No. of Blocks in MM : $64 / 4 = 16$



64 words

$$\log_2 64 = \log_2 2^6 \\ = 6 \text{ bits}$$

0	0	1	2	3
1	4	5	6	7
2	8	9	10	11
3	12	13	14	15
4	16	17	18	19
5	20	21	22	23
6	24	25	26	27
7	28	29	30	31
8	32	33	34	35
9	36	37	38	39
10	40	41	42	43
11	44	45	46	47
12	48	49	50	51
13	52	53	54	55
14	56	57	58	59
15	60	61	62	63



Physical Address

0	1	2	3	0
4	5	6	7	1
8	9	10	11	2
12	13	14	15	3
16	17	18	19	4
20	21	22	23	5
24	25	26	27	6
28	29	30	31	7
32	33	34	35	8
36	37	38	39	9
40	41	42	43	10
44	45	46	47	11
48	49	50	51	12
52	53	54	55	13
56	57	58	59	14
60	61	62	63	15

Physical Address Space

16 Blocks

$$\log_2 16 = \log_2 2^4$$
$$= 4 \text{ bits}$$

$$\log_2 64 = \log_2 2^6 = 6 \text{ bits}$$

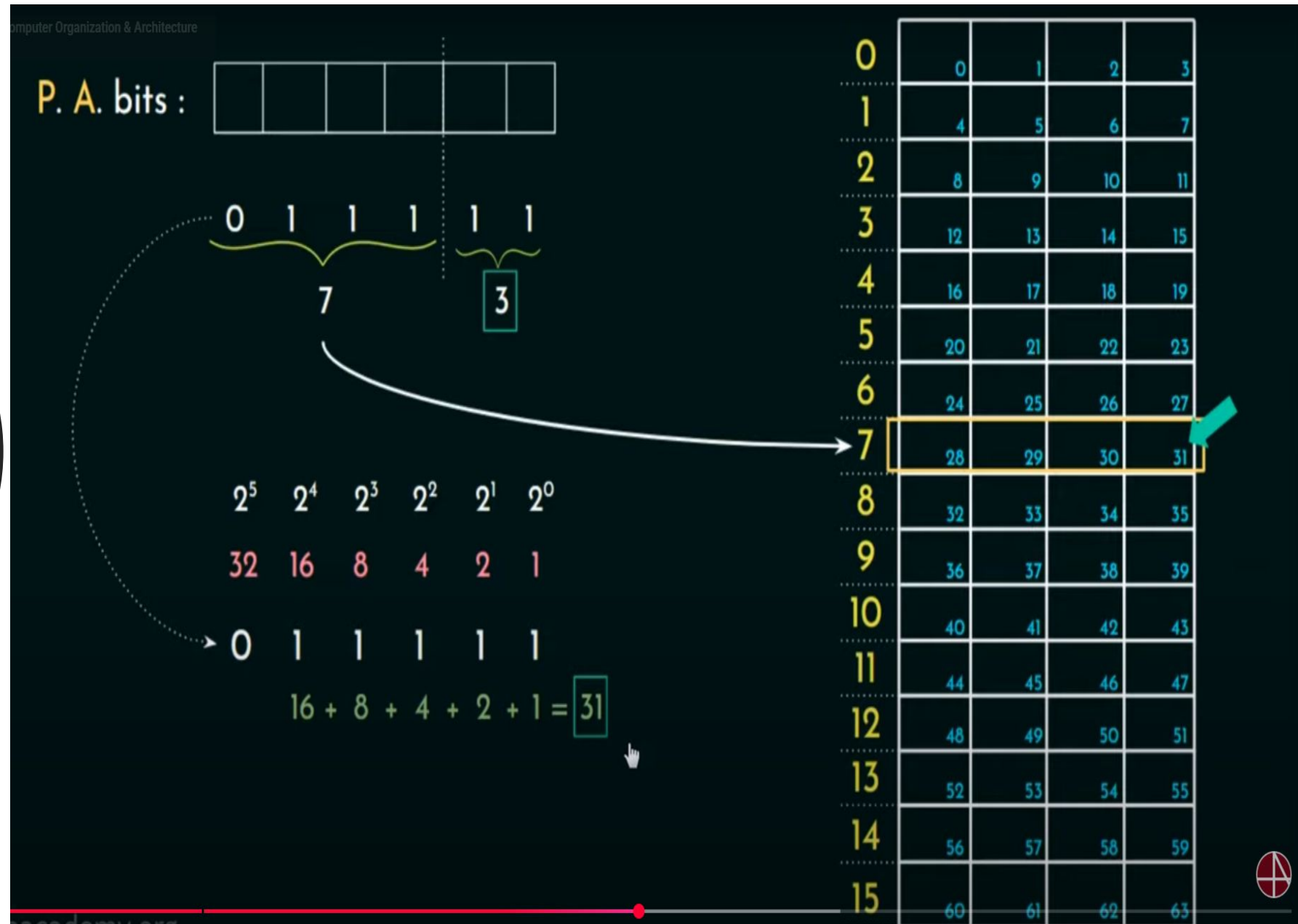
P. A. bits :

(Physical Address bits)



0 0 → 0th
0 1 → 1st
1 0 → 2nd
1 1 → 3rd

Example



Cache

Cache Size : 16 words

Line Size : 4 words

No. of Lines in Cache : $16 / 4 = 4$ i.e. 0, 1, 2, 3

Block Size : 4 words

Block Size = Line Size

4 Lines

$$\log_2 4 = \log_2 2^2$$

$$= 2 \text{ bits}$$

□ □

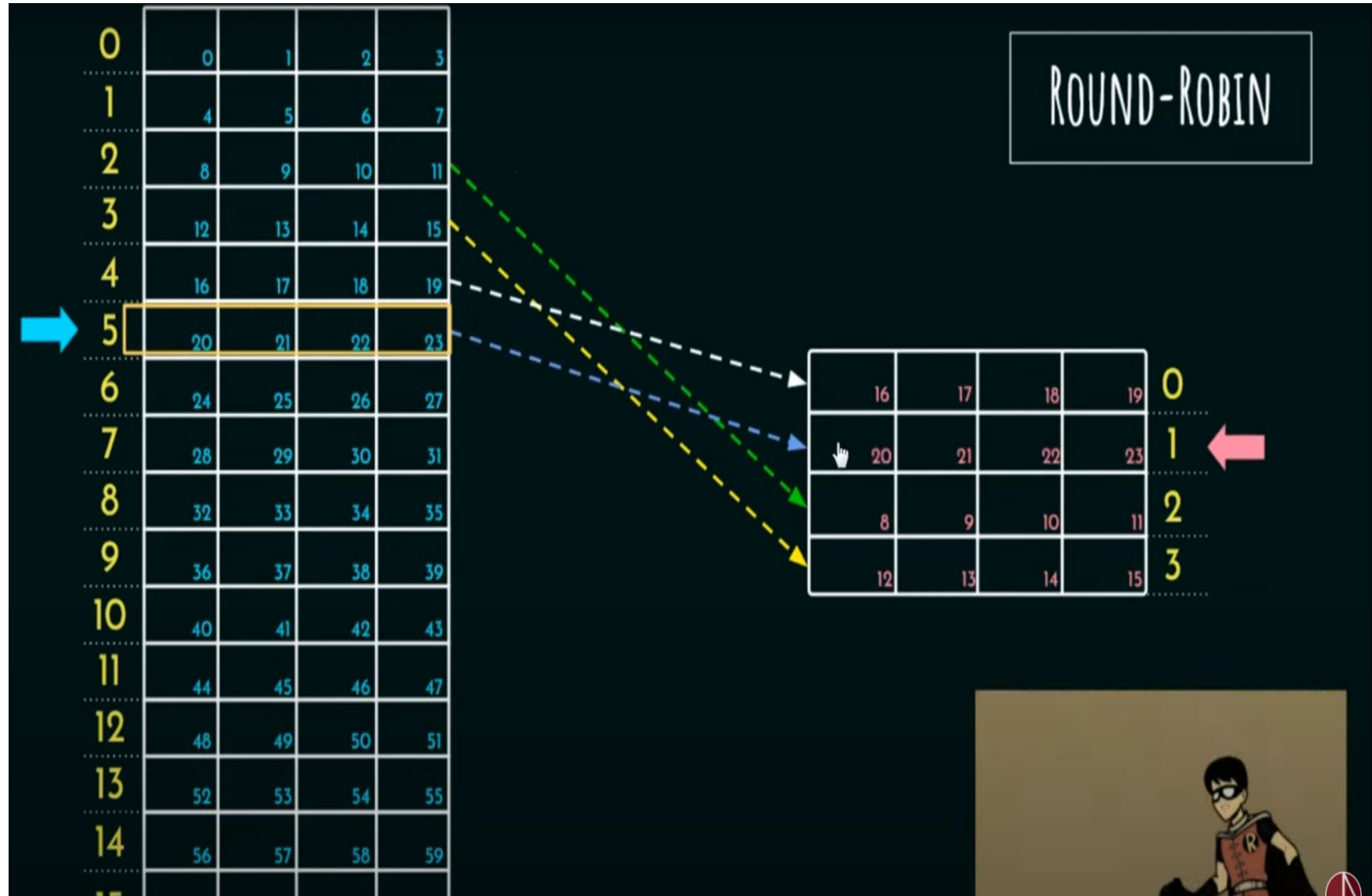
0 0 → 0

0 1 → 1

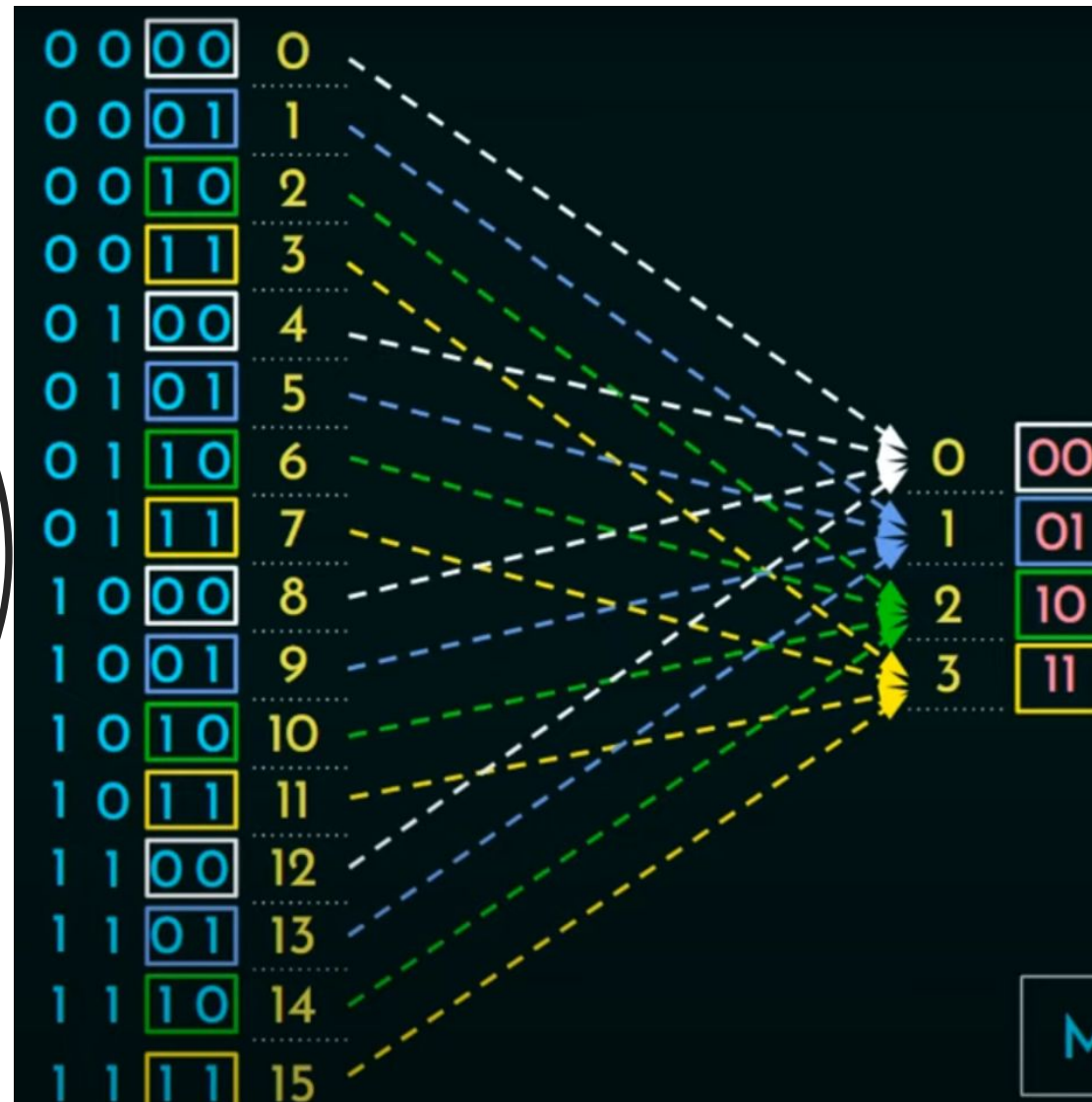
1 0 → 2

1 1 → 3

Mapping Memory to cache



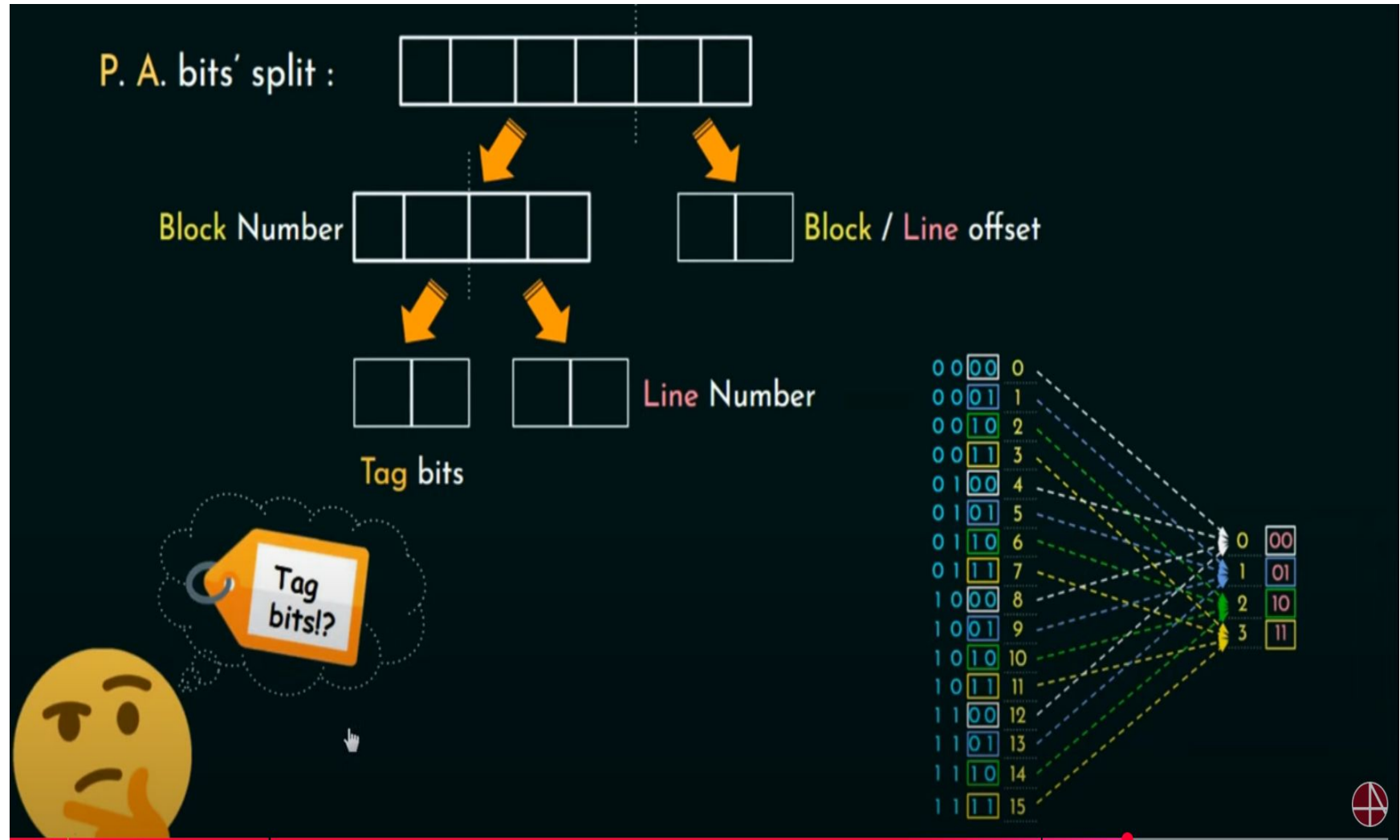
Mapping Memory to cache



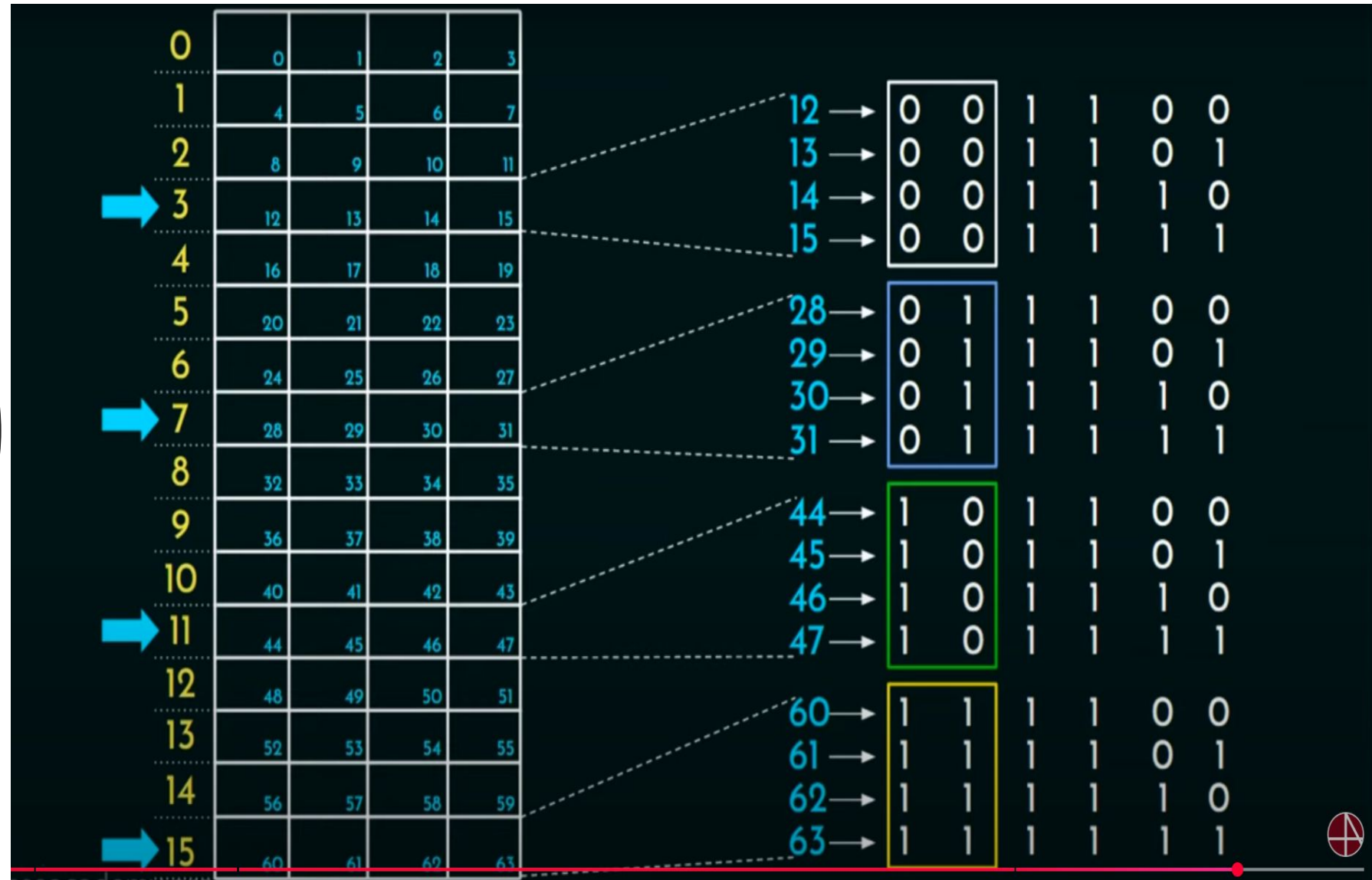
Many to One Relation



Tag Bits



Tag Bits



Replacement Algorithm

Cache has limited size → cannot hold all memory blocks at once.

When a new block needs to be loaded but cache is full → one of the existing blocks must be removed.

Replacement algorithm decides *which block to evict*.

Goal → maximize **hit ratio** and minimize **miss penalty**.

Common strategies include FIFO, LRU, and Random replacement.

Replacement Algorithm

FIFO (First-In, First-Out):

- The block that entered the cache *earliest* is replaced first.
- Works like a queue.

LRU (Least Recently Used):

- The block that has not been accessed for the *longest time* is replaced.
- Based on recent usage history.

Random Replacement:

- A cache block is chosen *at random* for replacement.
- Simple but unpredictable.

Write Policy

Determines **how data is written** to cache and main memory.

Decides **when main memory should be updated** after a CPU write.

Helps manage **speed vs. data consistency** between cache and main memory.

Two main types: **Write-Through** and **Write-Back**.

Write-Through



Data is written to **both cache and main memory immediately.**



Keeps main memory **always up-to-date.**



Simple to implement but **slower writes** due to frequent memory updates.

Write-Back



Data is written **only to the cache initially**.



Cache block is marked **dirty** if modified.



Main memory is updated later, only when the dirty block is replaced.



Faster and reduces memory traffic but slightly **more complex**.